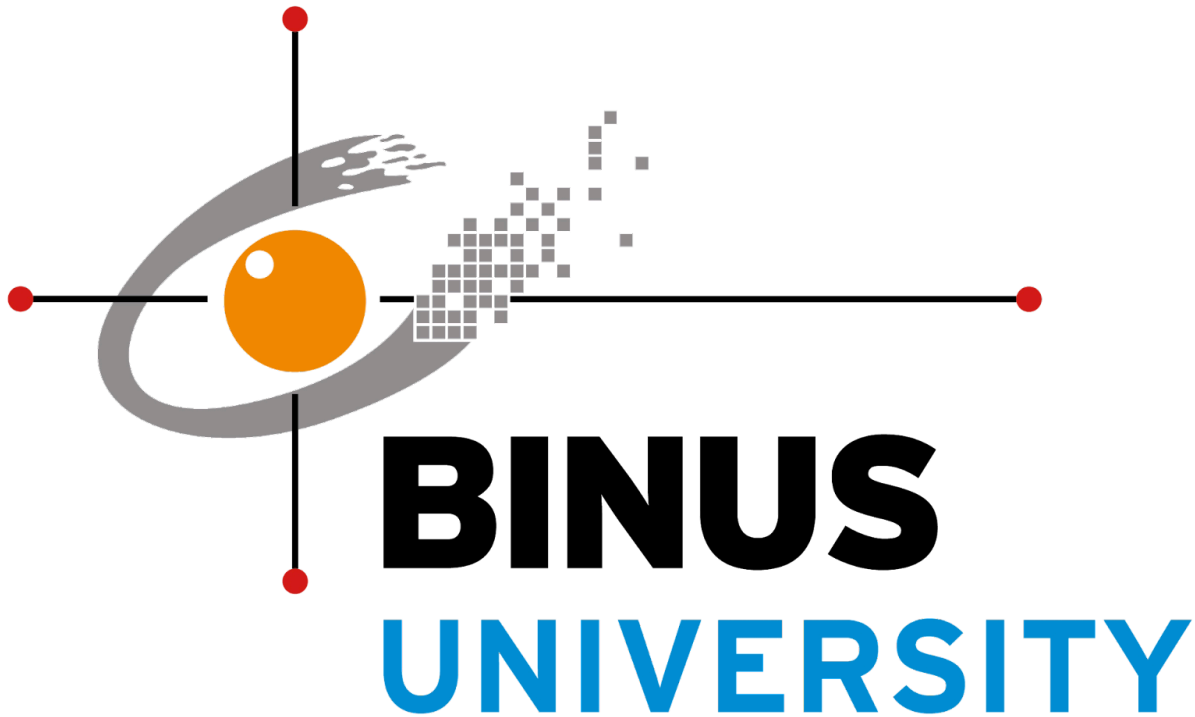


Data Structures & OOP

Final Project Report



Odyssey

Made By:

Louis Christian Dharma Panna Wibowo (2902731990)

Nizar Ramadhan (2902725672)

Kayla Isabelle Limaran (2902728081)

2026

Table of Contents

CHAPTER 1	3
Project Specifications	3
I. Background	3
II. Problem Description	3
CHAPTER 2	4
Solutions	4
A. Solution 1: Linked List	4
B. Solution 2: Binary Heap	4
C. Solution 3: ArrayList	4
CHAPTER 3	5
Experimentation & Complexity	5
Runtime Measurement & Comparison Table	5
a. Insertion at the front	5
b. Insertion in the Middle	6
c. Insertion at the End	7
Growth Graph	8
A. Insertion at the Front	8
B. Insertion at the Middle	9
C. Insertion at the End	10
Complexity Analyses	11
CHAPTER 4	12
Conclusion	12
CHAPTER 5	13
Program Screenshots and Class Structure	13
UML Class Diagram	13
Documentation	13
REFERENCES	18
APPENDIX	20
Program Manual	20
Git Repository Link	20

CHAPTER 1

Project Specifications

I. Background

Many hospitals have waiting areas without a clear system in place to manage and prioritize patients efficiently and legibly. It is also especially chaotic in stressful environments, such as emergency rooms and waiting areas. Our project, named Odyssey, aims to create a system that can reorder and help patients get treatments quickly and efficiently, while also providing doctors with clear, automatically prioritized data. With this system, we believe it would greatly benefit hospitals and allow staff to work as quickly as possible without the stress of identifying and prioritizing patients.

We chose to create a project surrounding this topic because we think it is important and necessary for healthcare to be effective and efficient for patient safety. It is also meant to help healthcare workers by minimizing the stress that comes from prioritizing and identifying patients in the hopes that this will assist them in focusing more time on providing care for their patients.

With this goal, our possible solution is to create a website or application that can collect data and information entered by users and automatically prioritize them based on symptom severity for doctors to see and use directly. We plan on using concepts in data structures and object-oriented programming to create our product because these concepts provide a basis for our goal in this project.

II. Problem Description

In emergency rooms, there are systems to prioritize patients based on severity to help quickly identify and treat high-priority patients first. However, this system is usually manual and can become very tedious, unclear, and messy, which can cause confusion and often cause mistakes and bias, which is why our group believes that a digital database would benefit these systems and hospitals that need them. This would make the experience much more efficient and clean for both patients and hospital staff, and would also alleviate some of the stress and workload within the emergency room.

There have been many cases where healthcare workers misinterpret symptoms patients define because of time constraints and bias, which can become dangerous for patients who require care quickly and urgently. This causes unnecessary suffering for the patients and hospital staff to be unorganized at times and become spread thin across many patients.

We do not want patients to compromise their health, and we do not want caretakers to be overworked, both of which are becoming the main issues in many hospitals that can cost lives, suffering, and unhealthy working conditions. These reasons become the main problem across many hospitals around the world that do not have proper funding and work ethic to keep staff healthy and patients safe. This motivated us to create our project to address this problem.

CHAPTER 2

Solutions

This project requires a method that can be implemented efficiently and can sort data while also tracking its contents. From this, we have chosen 2 possible solutions which will be compared with the required criteria for the best possible outcome.

A. Solution 1: Linked List

A linked list is a type of linear data structure where individual items are not necessarily at contiguous locations. The individual items are called nodes and connected using links.

The way this method would work would be as follows:

- Each patient is a node in the linked list structure
- The wait queue is unidirectional
- When a new patient enters, their symptoms are calculated based on severity, and they will be placed according to the location of the list
- Provides a rigid chronological ordering for all incoming patients

However, using a linked list can become inefficient because whenever an insertion occurs, depending on patient severity, searching it must use $O(N)$.

B. Solution 2: Binary Heap

A Binary Heap is a heap data structure that takes the form of a special type of complete binary tree, meaning all levels are filled except possibly the last, which is filled from left to right. Binary heaps are a common way of implementing priority queues. It allows fast access to the minimum or maximum element. There are two types of binary heaps: Min Heap and Max Heap. Min Heap means the value of the root node is the smallest, and Max Heap means the value of the root node is the largest.

The way this method would work would be as follows:

- Patient severity scores create heap nodes
- Insertion and extraction are all $O(\log n)$
- The highest priority patient is always at the top

An advantage of this method is that it provides guaranteed efficiency and logarithmic times for both adding new patients and getting the most urgent one, regardless of queue size. However, if a patient's severity changes or must be updated, a search must be performed with $O(n)$, which does not guarantee First In First Out (FIFO) for identical severity.

C. Solution 3: ArrayList

An ArrayList is a version of an array that is resizable and can grow or shrink dynamically as elements are added or removed. This data structure maintains the exact insertion order of elements and allows random and quick access at any index position.

The way this method would work would be as follows:

- Patient severity is added
- Select based on the highest severity

This method may have drawbacks, however, because in large numbers, searching may take time.

CHAPTER 3

Experimentation & Complexity

To experiment and determine which data structure will be most effective in this system, each data structure will undergo a runtime calculation based on the system nanotime. This method allows an objective and conclusive result in determining which data structure allows the fastest and most efficient runtime based on the different operation types. The calculation will be done using the start nanotime, the method, and the end nanotime. The difference between the two times will be the time it took to complete the operation.

Runtime Measurement & Comparison Table

a. Insertion at the front

Operation Type	Data Structure	Input Size	Runtime (Milliseconds)
Insertion at the Front	Linked List	1	0.1901
Insertion at the Front	Binary Heap	1	0.1914
Insertion at the Front	Array List	1	0.1943
Insertion at the Front	Linked List	10	0.2041
Insertion at the Front	Binary Heap	10	0.2763
Insertion at the Front	Array List	10	0.5833
Insertion at the Front	Linked List	100	0.521
Insertion at the Front	Binary Heap	100	0.7341
Insertion at the Front	Array List	100	0.8403
Insertion at the Front	Linked List	1,000	1.6321
Insertion at the Front	Binary Heap	1,000	1.9628
Insertion at the Front	Array List	1,000	2.4193
Insertion at the Front	Linked List	10,000	4.8969
Insertion at the Front	Binary Heap	10,000	6.2588
Insertion at the Front	Array List	10,000	7.2178
Insertion at the Front	Linked List	100,000	21.4307
Insertion at the Front	Binary Heap	100,000	31.3932
Insertion at the Front	Array List	100,000	40.1381

b. Insertion in the Middle

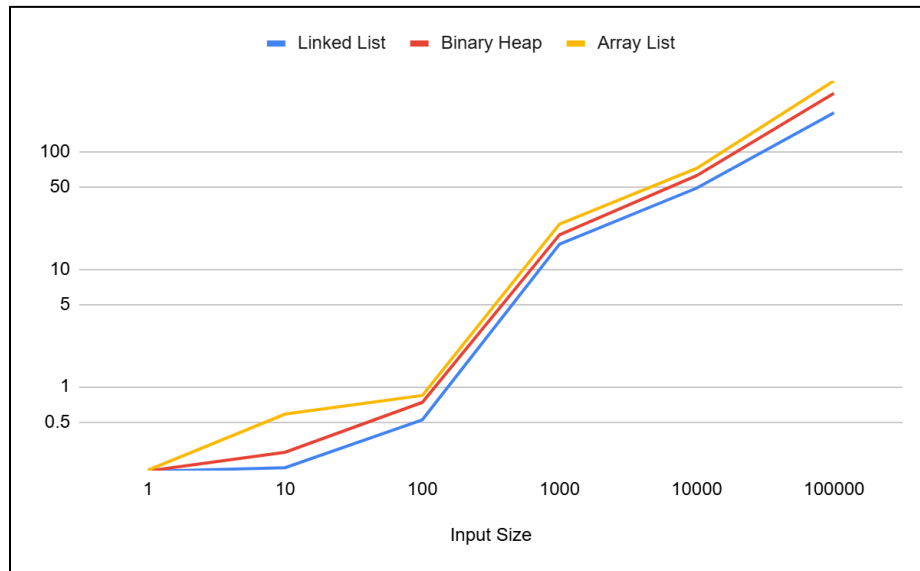
Operation Type	Data Structure	Input Size	Runtime (Milliseconds)
Insertion in the Middle	Linked List	1	0.1901
Insertion in the Middle	Binary Heap	1	0.1914
Insertion in the Middle	Array List	1	0.1943
Insertion in the Middle	Linked List	10	0.6452
Insertion in the Middle	Binary Heap	10	0.2492
Insertion in the Middle	Array List	10	0.6912
Insertion in the Middle	Linked List	100	0.8435
Insertion in the Middle	Binary Heap	100	0.4788
Insertion in the Middle	Array List	100	1.0732
Insertion in the Middle	Linked List	1,000	2.8511
Insertion in the Middle	Binary Heap	1,000	1.6408
Insertion in the Middle	Array List	1,000	3.4103
Insertion in the Middle	Linked List	10,000	54.0283
Insertion in the Middle	Binary Heap	10,000	5.7413
Insertion in the Middle	Array List	10,000	59.2938
Insertion in the Middle	Linked List	100,000	14932.9462
Insertion in the Middle	Binary Heap	100,000	22.3489
Insertion in the Middle	Array List	100,000	17302.2049

c. Insertion at the End

Operation Type	Data Structure	Input Size	Runtime (Milliseconds)
Insertion at the End	Linked List	1	0.1901
Insertion at the End	Binary Heap	1	0.1914
Insertion at the End	Array List	1	0.1943
Insertion at the End	Linked List	10	0.8734
Insertion at the End	Binary Heap	10	0.2441
Insertion at the End	Array List	10	0.9143
Insertion at the End	Linked List	100	0.9217
Insertion at the End	Binary Heap	100	0.4867
Insertion at the End	Array List	100	0.981
Insertion at the End	Linked List	1,000	4.2959
Insertion at the End	Binary Heap	1,000	1.5218
Insertion at the End	Array List	1,000	4.6029
Insertion at the End	Linked List	10,000	94.1519
Insertion at the End	Binary Heap	10,000	5.0438
Insertion at the End	Array List	10,000	96.1039
Insertion at the End	Linked List	100,000	22265.6716
Insertion at the End	Binary Heap	100,000	24.5597
Insertion at the End	Array List	100,000	25123.3929

Growth Graph

A. Insertion at the Front

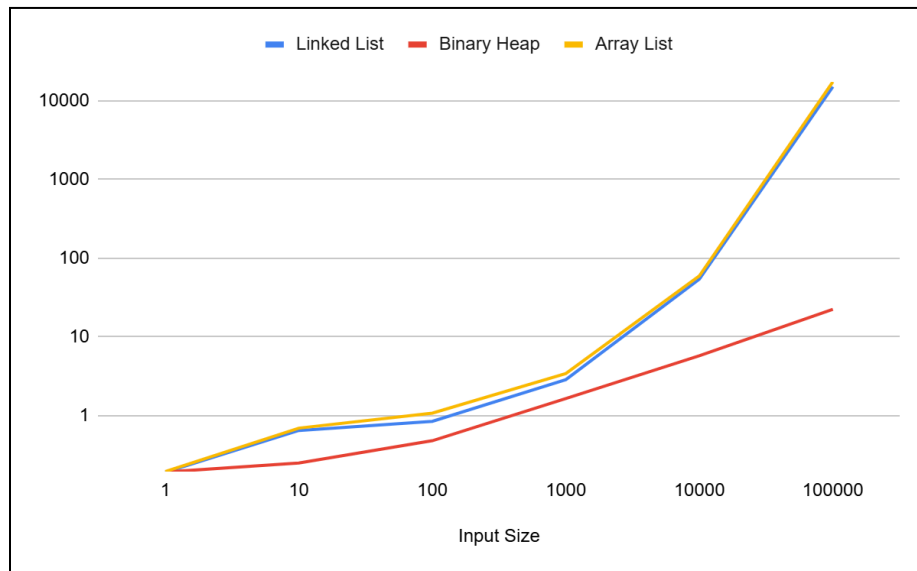


From front insertion, the linked list performs the best. This makes sense because inserting at the front of the linked list only needed the change of the pointer head. It doesn't need to shift to another element. So, the complex theories commonly have $O(1)$ per insertion. The binary heap slightly slowed because it is not only inputting data in the "front". A heap inputs new data, then arranges itself by heapifying up to maintain its priority structure. By this, it gives complexity a $O(\log n)$

ArrayList is the slowest in this graph because the insertion at the front must include all elements that exist to shift one position to the right. By this, it creates an insertion in the front part that becomes $O(n)$. So, when the size of the input becomes bigger, ArrayList needs more time than the other two.

Based on the result, with the input size of 100,000, the linked list is the fastest with a time of 21,430.7 ms, the binary heap is 31,393.2 ms, and the ArrayList is the slowest one with a time of 40,138.1 ms

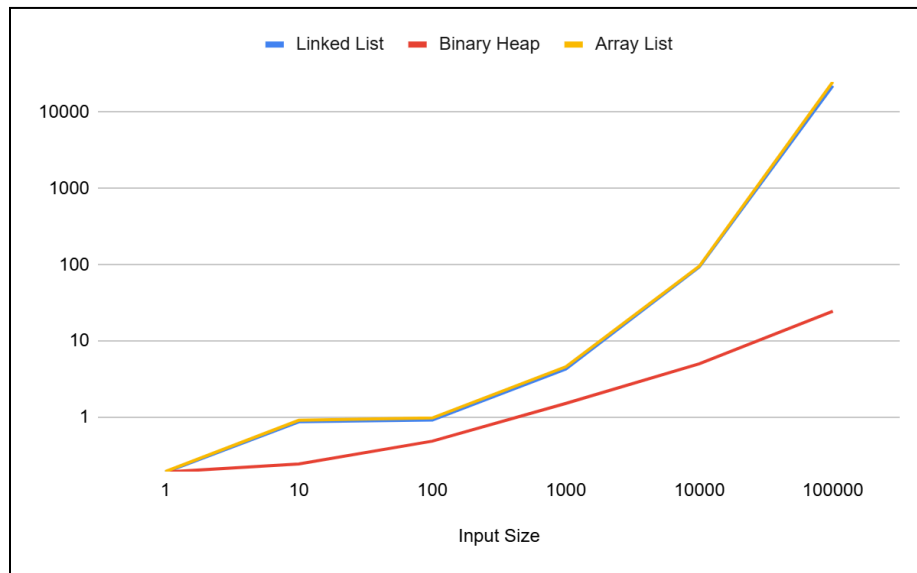
B. Insertion at the Middle



For the middle insertion, the graph changes significantly. Binary heap becomes the best structure because it only needs time $O(\log n)$ to insert and recover the heap structure. It doesn't need to find the middle position manually, like in the list. Linked lists become slower because when inserting in the middle, the program must first travel through the list to find the middle position. Traversing the list takes $O(n)$ time. The actual insertion after reaching the node is fast, but finding the position is the expensive part.

The array list became slower because middle insertions required shifting more elements after the insertion point. This thing could give complexity $O(n)$. This is why the Binary Heap grows slowly, while the Linked List and the ArrayList increase sharply. For the input size 100,000 Binary Heap only grows until 22,3489 ms. while the Linked List reached 14932,9462 ms, and for the Array List, it took 17302,2049 ms

C. Insertion at the End



For the final insertion, a binary heap is the most stable graph. Its insertion remains $O(\log n)$ because it's replaced with a new value inside the heap and later reorganizes the trees if it is needed.

Based on the result, the complexity of the linked list increased sharply. This shows that the implementation based on the graph possibly needs to find from head to end before doing the insertion, which can become $O(n)$. If the Linked List has a pointer to the tail directly, the insertion end can be $O(1)$. At 100,000 input size, the Binary Heap just needs time for 24.5597 ms. while the Linked List needed time for 22265,6716 ms, and the Array List needed time for 25123,3929 ms. This shows the conclusions that the Binary Heap is the most efficient structure for the entry System Hospital, based on this priority

Complexity Analyses

To find the best possible solution, our team conducted experiments on each data structure with the goal of finding a method that has the best time and space complexity and can handle large amounts of data with the shortest runtime. To do this, data was going to be inserted at the front, middle, and end of each of the structures, with an increasing amount of data put in, ranging from 1 to 100000.

When inserting data to the front, it is seen that the Linked List has the shortest runtime of the 3 due to its $O(1)$ complexity because it only needs to change the pointers rather than shifting the elements one by one. Binary Heap is slower than a Linked List with its $O(\log n)$ complexity because every insertion is shifted through elements to keep the structure balanced. ArrayList has the slowest runtime with its $O(n)$ complexity because it must compare priorities with every element to properly shift the data.

Additionally, the difference in runtime per insertion is seen to be quite minimal as the size increases. Linked List timed at 0.1901ms and remained consistently the shortest throughout the testing, compared to Binary Heap and Array List at 0.1914ms and 0.1943ms. This difference stayed relatively steady up to size 100,000, finally reaching 21.4307ms, 31.3932ms, and 40.1381ms, which has the largest difference of approximately 10 to 20 milliseconds between each result.

While inserting data into the middle, the Binary Heap is shown to have a large, drastic difference, with it having the shortest and steadiest incline in runtime despite the increased input. This is due to the Binary Heap's resulting $O(\log n)$ complexity, thus not needing to search for the middle of the data structure for insertion, and simply sifting through the data. Linked List and Array List are not as effective in keeping the runtime low, due to their resulting in $O(n)$ complexity. Due to them requiring finding the middle value first for insertion, it requires constant shifting through the entire list, and takes a lot of power and resources. Linked List remained the fastest at size 1 with a time of 0.1901ms, 0.1914ms, and 0.1943ms. However, this changes drastically with the Binary Heap increasing steadily, while the Linked List and the ArrayList spike per input size.

For insertion at the end, the Binary Heap is shown to have similar results with inserting at the middle, having huge differences from the rest. It also has the shortest and steadiest incline in runtime despite the increased input remaining in $O(\log n)$. Linked List and Array List continue to be $O(n)$ and ineffective in keeping the runtime low compared to the Binary Heap, which is designed to be balanced from the start. It is also seen that at input size 100,000, it changes to $O(1)$ instead. The runtime difference is also much higher where input size 100,00 ends, with Binary Heap timed at 24.5597ms, while Linked List and Array List at 22265.6716ms and 25123.3929ms. Binary Heap becomes the fastest and most gradual, while ArrayList is the slowest of the three.

From these tests, we are able to conclude that a binary heap is the best possible solution that our project could use. The experimentation also showcases that ArrayList provided the worst outcome of the three and highlighted its major flaws that brought our team to the decision against using it entirely.

CHAPTER 4

Conclusion

Our goal with this project is to create a program that is able to collect data and sort it based on a set severity scale efficiently, which will then be reviewed and listed for doctors to see and use to quickly maneuver through large numbers of patients. We had also created a design that was meant to be simple and accessible for both doctors and patients to use, which would allow them to insert their data without difficulty. This program relies on certain data structures that would be able to do these specific functions in scale with minimal runtime, and because of this, we had chosen 3 different types to compare: Linked List, Binary Heap, and ArrayList.

To find the best structure to use, we conducted a series of experiments that tested the amount of data they were able to handle and the time it took. With every increase in input size, the runtime also increases, and depending on the data structure, the runtime will vary significantly. Using SQL, we were able to get the run time per experiment. While testing our program with each of the methods, it was found that the Binary Heap is the most efficient and effective structure because of its process that allows data to sift through cleanly and efficiently. The Binary heap was able to keep up with large amounts of data with only a gradual increase in time compared to the others, and because of this reason, we can conclude that this method is the best for our project and was implemented. However, the testing has also shown us the inefficiency of using ArrayList and therefore was no longer considered as an option.

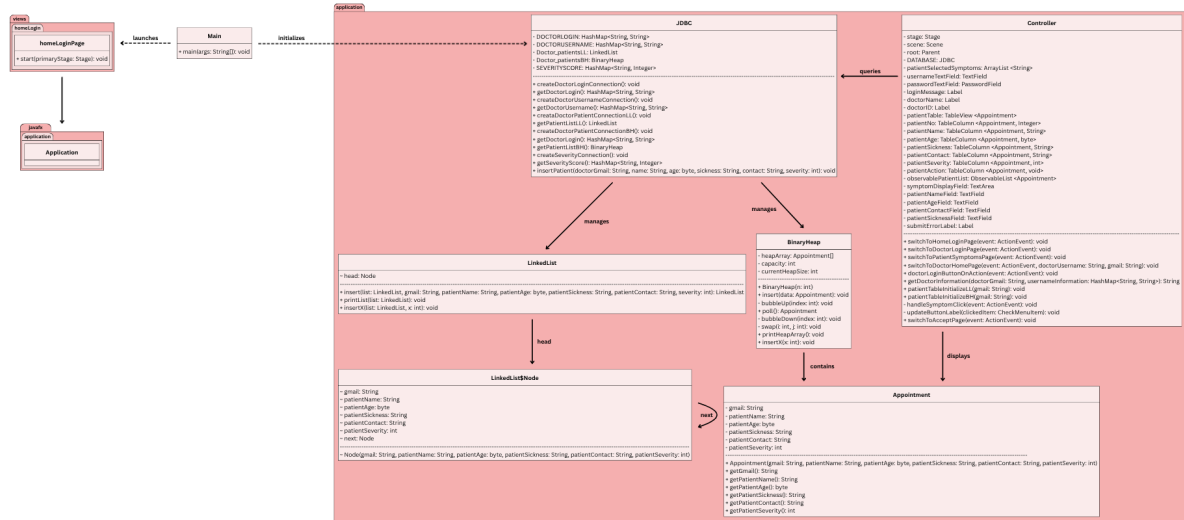
Using the program with the implemented structure, a patient is able to enter their specific information, including name, age, contact information, and specific descriptions of the patient's symptoms in the form of written questions with dropdown options. Once the patient submits, their data gets inserted into a database that uses a binary heap to rearrange each patient based on the severity score, which then would be placed in a list that doctors can access. Doctors are able to log in to their profile and see this list to review and track their patients in person.

Taking everything into account, it is seen that the Binary Heap is able to benefit greatly in making this project. It is able to prioritize quickly despite the large amount of incoming inputs and size. It is also able to insert and search for data easily without taking too much time, which is a great advantage in an emergency room setting where everything must be done fast and efficiently. This would also make the experience easier for patients to use because of its simple design, and allows hundreds of patients to check in at once.

Using our project, hospitals would be able to save and treat patients much faster without having to prioritize and manage manually. They would not need to take time to analyse and sort through patients, and would reduce existing bias entirely. It would make emergency rooms much safer for both patients' health and staff's efficiency, allowing both to be much healthier as well.

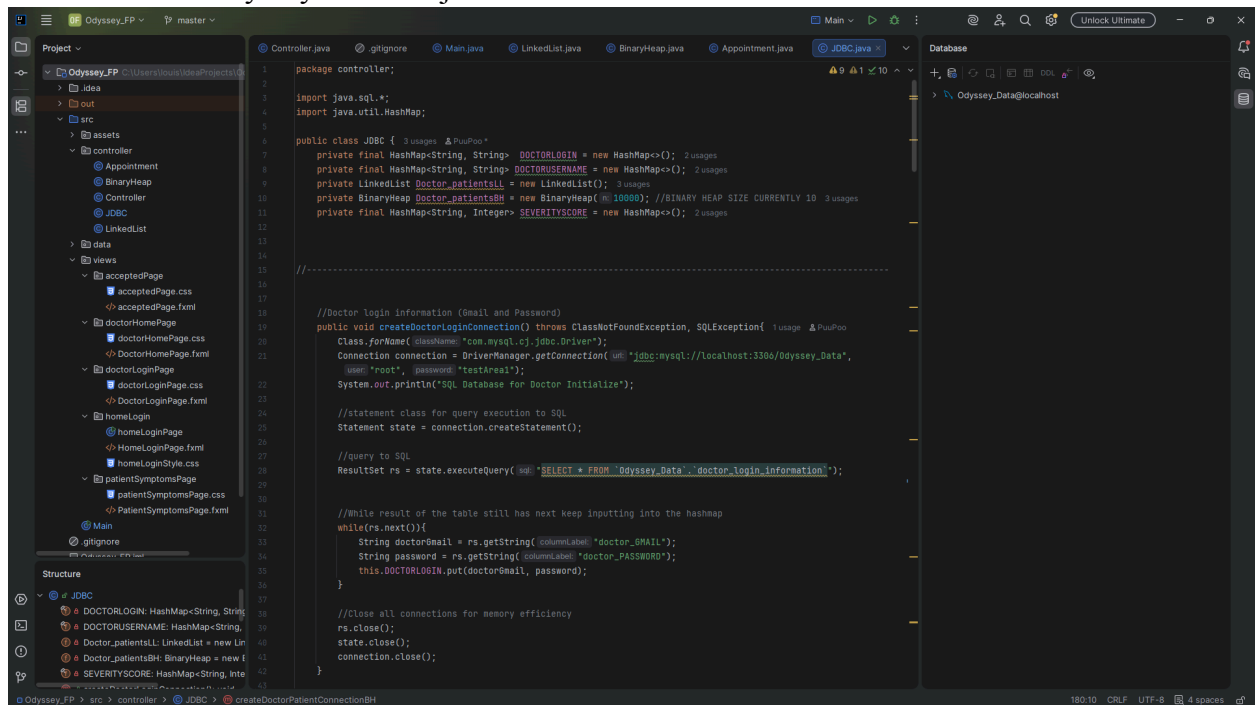
CHAPTER 5

UML Class Diagram

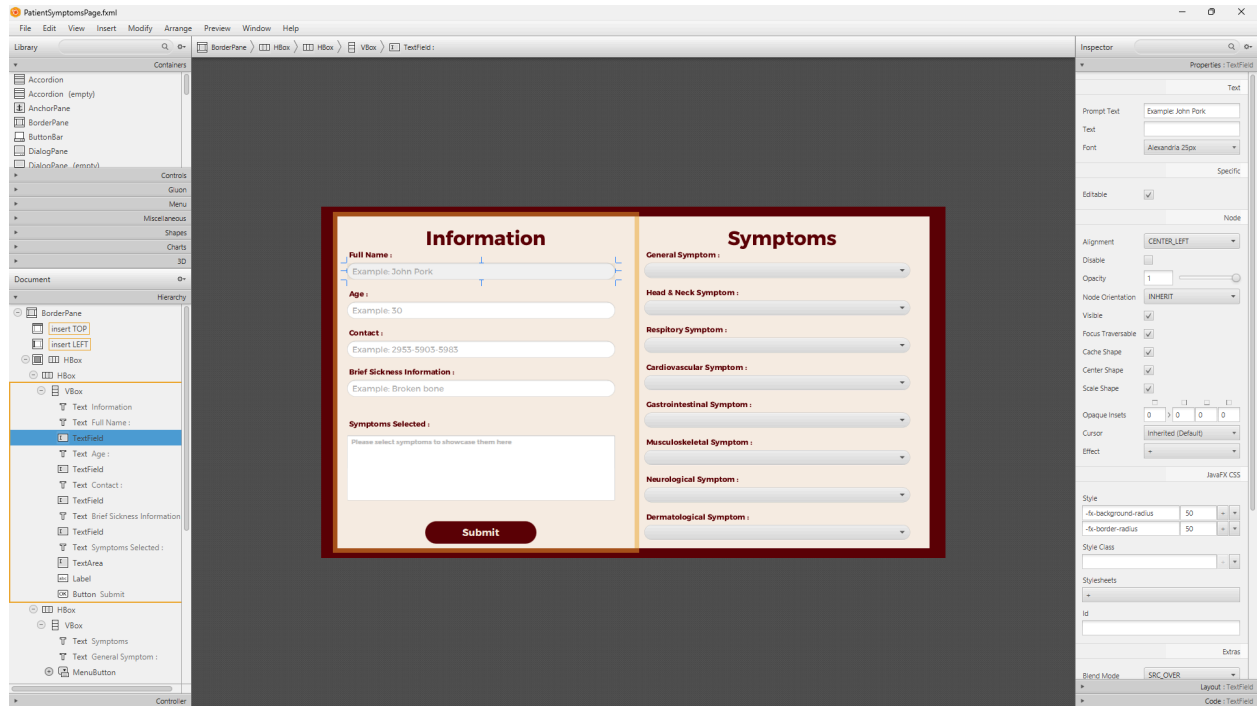


Documentation

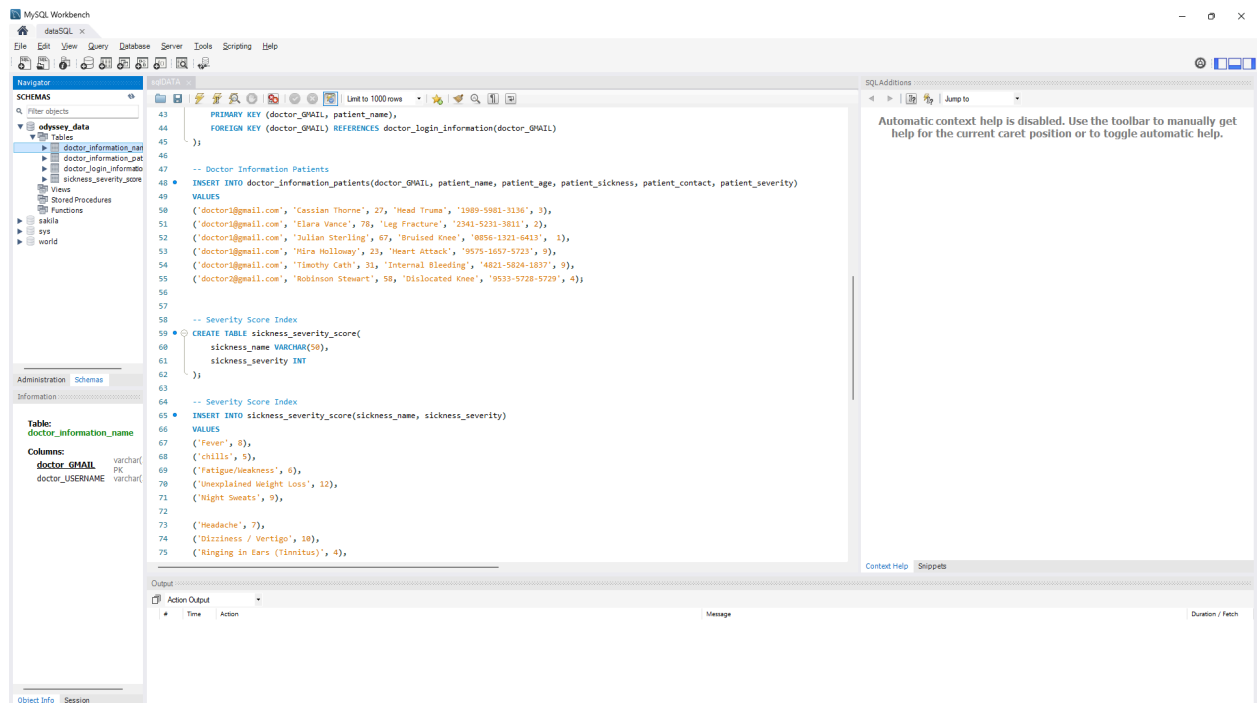
Screenshots below provide a quick preview of the application, visual appearance, and coding structure of the Odyssey Final Project.



(Code Structure and Hierarchy)



(Example of an FXML frontend view)



(Excerpt of the SQL database code)

WELCOME

Your health is our priority! By inputting your symptoms now, you help our doctors prepare for your visit. Patients with severe symptoms are prioritized to ensure everyone receives the right care at the right time.

Who are you?

Doctor

Patient

(Login Page/Main Home Page)

Information

Full Name :
Example: John Pork

Age :
Example: 30

Contact :
Example: 2953-5903-5983

Brief Sickness Information :
Example: Broken bone

Symptoms Selected :
Please select symptoms to showcase them here

Submit

Symptoms

General Symptom :

Head & Neck Symptom :

Respiratory Symptom :

Cardiovascular Symptom :

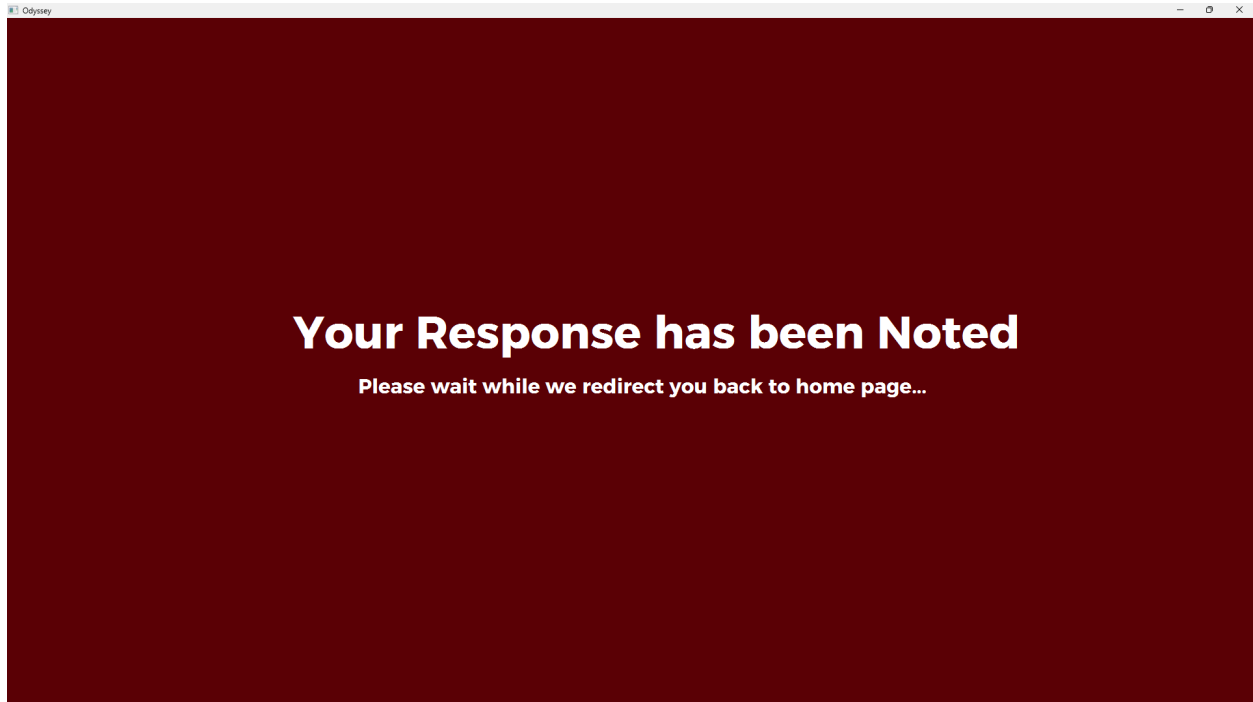
Gastrointestinal Symptom :

Musculoskeletal Symptom :

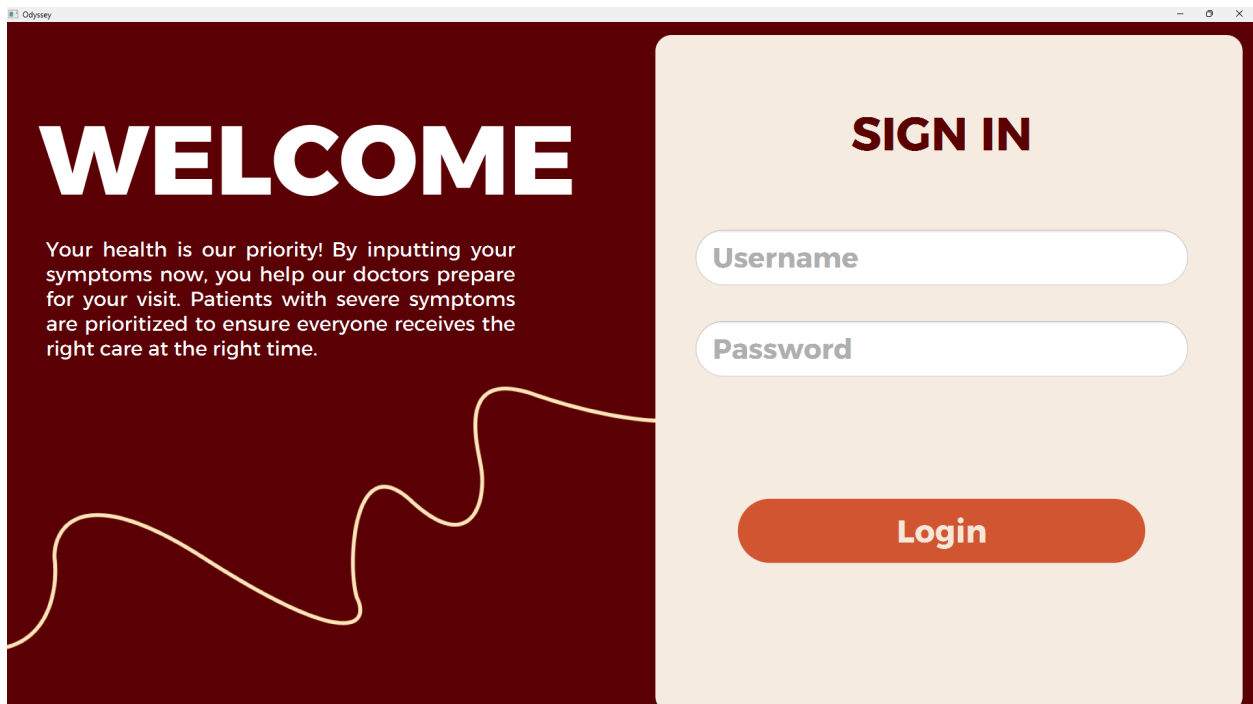
Neurological Symptom :

Dermatological Symptom :

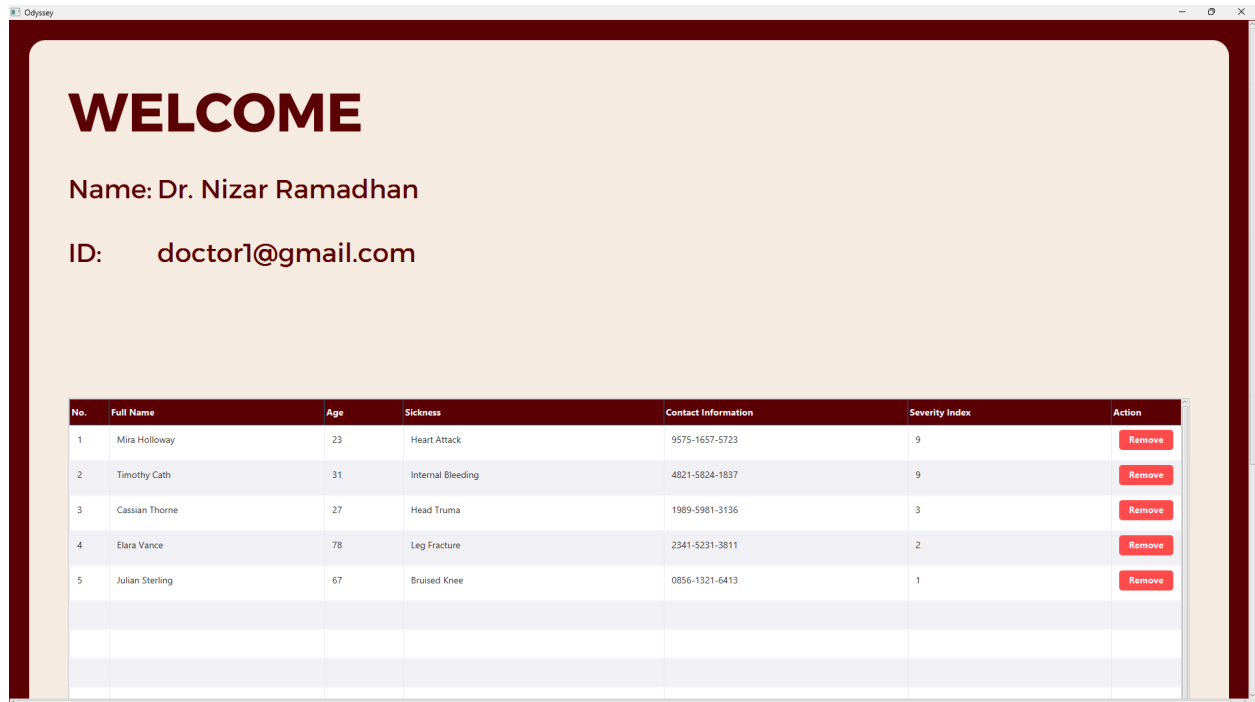
(Patient Page)



(Patient Page Accepted Page)



(Doctor Login Page)



(Doctor Home Page Example)

REFERENCES

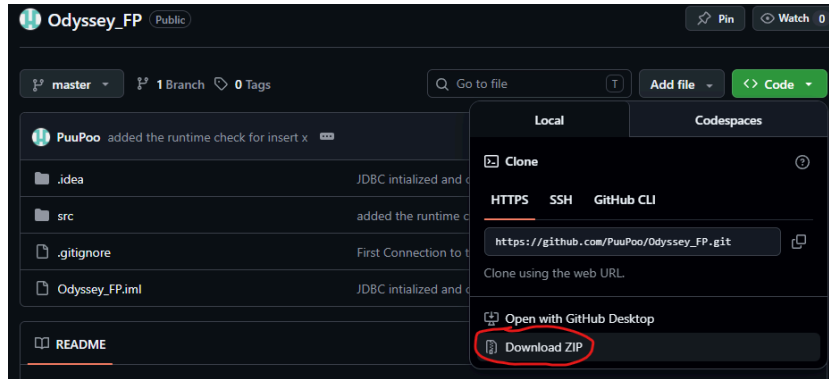
- Alex The Analyst. (2023, October 31). *Installing MySQL and Creating Databases | MySQL for Beginners*. Wwww.youtube.com. <https://www.youtube.com/watch?v=wgRwITQHszU>
- Bro Code. (2021, January 12). *JavaFX switch scenes* . Wwww.youtube.com. <https://www.youtube.com/watch?v=hcM-R-YOKkQ>
- Clever Coder. (2020, October 5). *Create A Beautiful Login Page using JavaFX*. YouTube. <https://www.youtube.com/watch?v=gT7RjdJvPis>
- Code Amir. (2020, April 19). *JavaFX Scene Builder Tutorial 39 - TableView and TableColumn*. Wwww.youtube.com. <https://www.youtube.com/watch?v=A5fQbsJ-iF8>
- Doudareva, E., & Carter, M. (2022). Discrete event simulation for emergency department modelling: A systematic review of validation methods. *Operations Research for Health Care*, 33, 100340. <https://doi.org/10.1016/j.orhc.2022.100340>
- GeeksforGeeks. (2014, November). *Binary Heap*. GeeksforGeeks. <https://www.geeksforgeeks.org/dsa/binary-heap/>
- Iwan Adhicandra, Safitri Nurhidayati, & Tribowo Rachmat Fauzan. (2024). Optimization of Hospital Queue Management Using Priority Queue Algorithm and Reinforcement Learning for Emergency Service Prioritization. *International Journal Software Engineering and Computer Science (IJSECS)*, 4(2), 512–522. <https://doi.org/10.35870/ijsecs.v4i2.2772>
- LearningFromExperience. (2024, December 3). *How to Connect MySQL Database in IntelliJ IDEA (Step-by-Step)*. YouTube. <https://www.youtube.com/watch?v=EFCuimS7VwI>
- Logic Lambda. (2021, November 5). *Implementing Heaps in Java - Part 1 - Introduction, HeapNodes, and LinkedHeaps*. YouTube. <https://www.youtube.com/watch?v=sj6QqonekkY>

- MySQL. (2019). *MySQL :: MySQL Workbench Manual*. Mysql.com.
<https://dev.mysql.com/doc/workbench/en/>
- Oracle. (2013a). *JavaFX Scene Builder User Guide: About This User Guide | JavaFX 2 Tutorials and Documentation*. Docs.oracle.com.
https://docs.oracle.com/javafx/scenebuilder/1/user_guide/jsbpub-user_guide.htm
- Oracle. (2013b, October 9). *Introduction to FXML | JavaFX 8.0*. Docs.oracle.com.
https://docs.oracle.com/javase/8/javafx/api/javafx/fxml/doc-files/introduction_to_fxml.html
- Ripples Code. (2017, November 12). *JAVAFX Tutorial 15-Button Events in Scene Builder*. YouTube. <https://www.youtube.com/watch?v=pzQXd08BPy4>
- Santoso, H. R., Novayani, W., Lestari, I., & Putri, A. (2024). QUEUE MANAGEMENT SYSTEM FOR COMMUNITY HEALTH CENTRE (CASE STUDY: PUSKESMAS UMBAN SARI). *Jurnal Komputer Terapan*, 10(2), 181–189.
<https://doi.org/10.35143/jkt.v10i2.6431>
- TapTap. (2023, August 17). *Connect MySQL to Java in IntelliJ 2024 | Full JDBC Tutorial for DB Connectivity*. YouTube. <https://www.youtube.com/watch?v=9ntKSLLDeSs>
- Telusko. (2017, December 22). *#5 Linked List Implementation in Java Part 1 | Data Structures*. YouTube. <https://www.youtube.com/watch?v=SMIq13-FZSE>
- tookootek. (2021, May 15). *JavaFX Tutorial | Login Screen with database connection (2021)*. YouTube. <https://www.youtube.com/watch?v=J0IE5LRyzx8>
- W3Schools. (n.d.). *DSA Linked Lists*. Wwww.w3schools.com.
https://www.w3schools.com/dsa/dsa_theory_linkedlists.php
- Yamin, M. (2018). IT applications in healthcare management: a survey. *International Journal of Information Technology*, 10(4), 503–509. <https://doi.org/10.1007/s41870-018-0203-3>

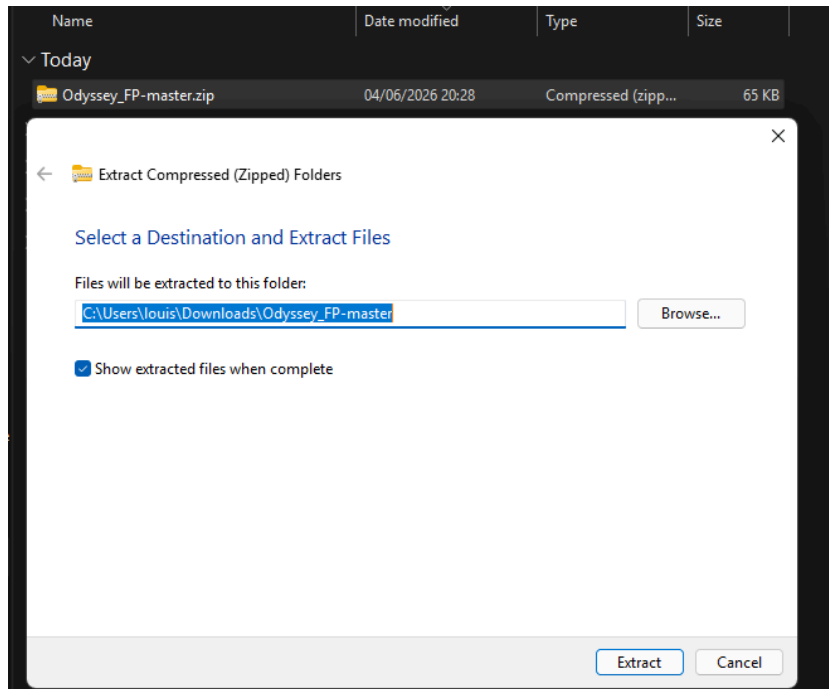
APPENDIX

Program Manual

1. Download the ZIP folder for the Odyssey project from the Git Repository Link below

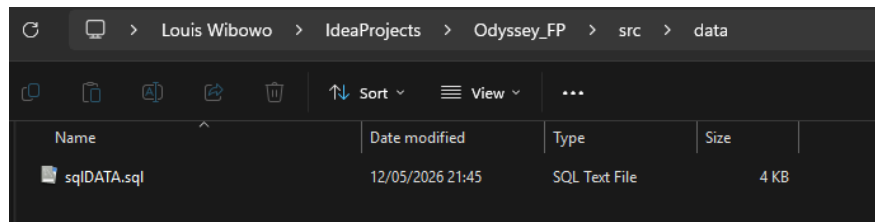


2. Extract all the files from the ZIP folder

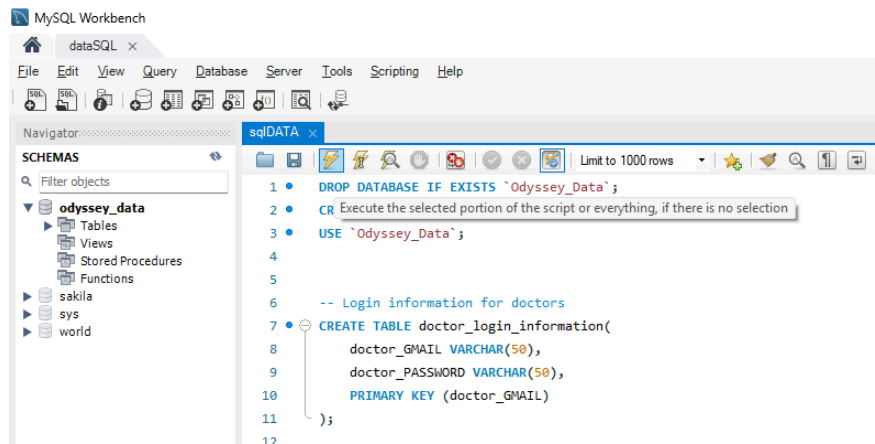


3. Ensure that your desktop and or working space has MySQL workspace, MySQL server, and JavaFX dependencies downloaded

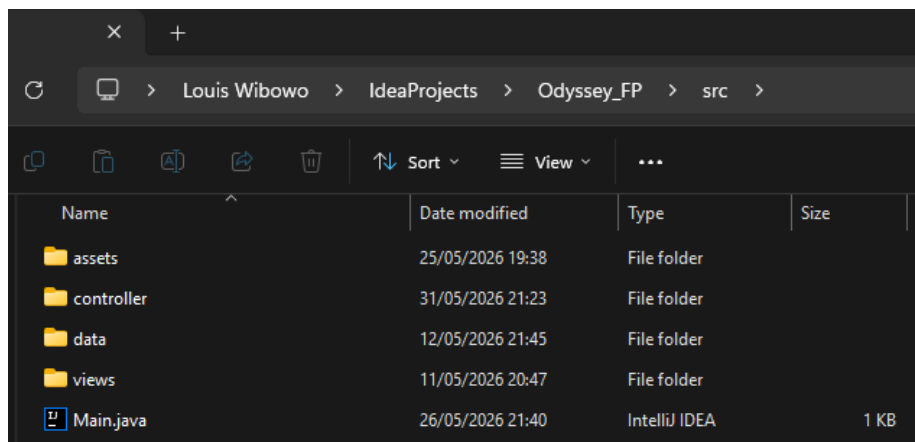
4. Open the following folder directories to access the data (Odyssey_FP > src > data)



5. Open the file and run the code in MySQL to load the data into the MySQL server



6. Open the following folder directories to access the application (Odyssey_FP > src)



7. Run the Main.java file
 - a. Information on login information can be viewed in the MySQL code

Git Repository Link

https://github.com/PuuPoo/Odyssey_FP